ITE183 Laboratory 2: WebSocket Server

Laboratory Report | Web Systems and Technologies

Student	**Gio Patrick G. Cimeni**
Section / Instructor	IT4D - Jofel A. Batutay
Date / Repository	27 October 2026 / https://github.com/giopatrick11/ITE183_Lab2_Websocket

I. Objectives

Implement an HTTP and WebSocket server using Bonezegei Scripting Language (BSL) v1.3.3; serve an interactive browser chat; manage client connection and disconnection events; broadcast messages to multiple active clients; and return a custom page for undefined routes.

II. Environment and Implementation

Environment: GitHub Codespaces (Linux), Chrome browser, Bonezegei Scripting Language v1.3.3, and port 8000.

The project contains app.bzg (server and WebSocket event handlers), package.json (package metadata and dependency), public/index.html (chat interface), public/404.html (custom missing-page response), .gitattributes (GitHub language highlighting), README.md, and an MIT license. The server sends a welcome message after connection and broadcasts client messages to all active users.

III. Procedure

1. Opened the project in GitHub Codespaces and verified the installed Bonezegei version using bonezegei -v.
2. Executed bzg install to install the HTTP dependency, followed by bzg run to start the server on port 8000.
3. Opened the forwarded port 8000 URL in two Chrome windows and exchanged messages to test real-time broadcasting.
4. Visited the undefined /test route to check the custom 404 page.
5. Inspected the browser WebSocket request in Chrome DevTools to check the protocol upgrade.

IV. Results and Discussion

The supplied screenshots show a successful Bonezegei v1.3.3 version check, HTTP dependency installation, and a running chat server. Two browser clients both display “Connected” and receive the same “Client 1: hehe” and “Client 2: hello” broadcasts. The undefined /test path displays the custom 404 page. Chrome DevTools also shows a wss:// WebSocket request with status 101 Switching Protocols. These observations support successful connection handling, multi-client broadcasting, page routing, and WebSocket handshake.

Note: The screenshots demonstrate the functions above, but they do not independently prove every implementation detail or all edge cases. A proper HTTP 404 status code, distinct from the 404 page content, was not verified in the supplied screenshots.

V. Conclusion

The laboratory demonstrated an event-driven WebSocket chat implemented with Bonezegei Scripting Language. The observed behavior confirms that multiple browser clients can maintain connections and receive messages broadcast by the server. The installation, server execution, WebSocket connection, and custom missing-page interface were also documented.

Evidence 1. Bonezegei version

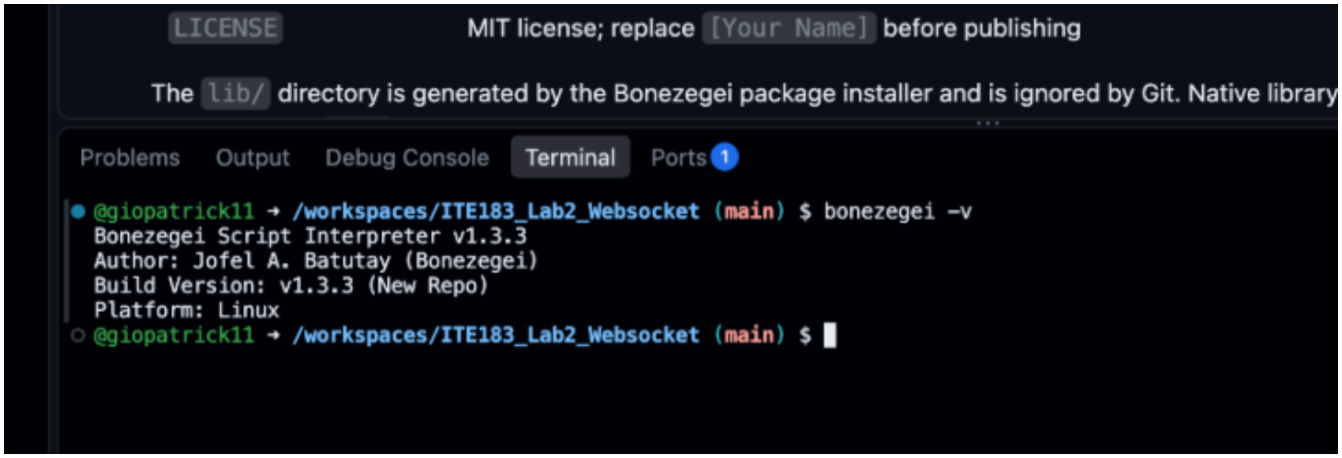


Figure 1. bonezegei -v output confirms Bonezegei Scripting Interpreter v1.3.3.

Evidence 2. Dependency installation and server startup



Figure 2. bzg install completes and bzg run starts the server; connection logs are visible.

Evidence 3. Multi-client message broadcast

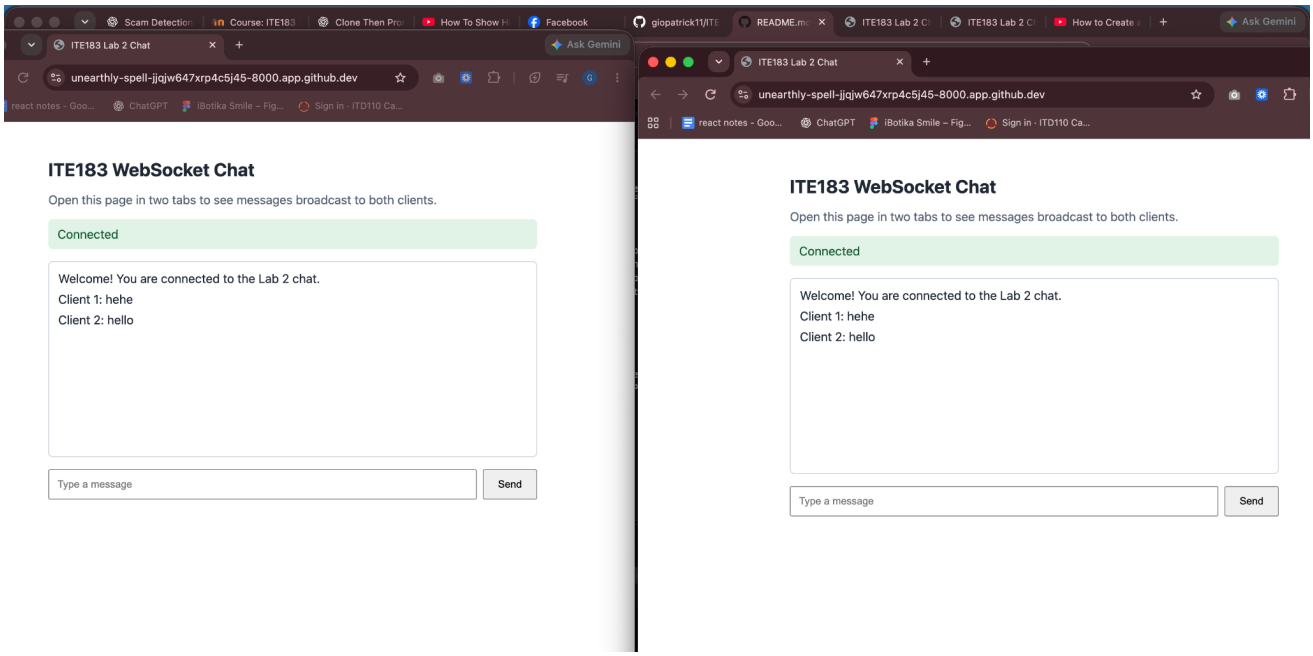


Figure 3. Two browser windows show “Connected” and receive the same messages from two clients.

Evidence 4. Custom 404 page

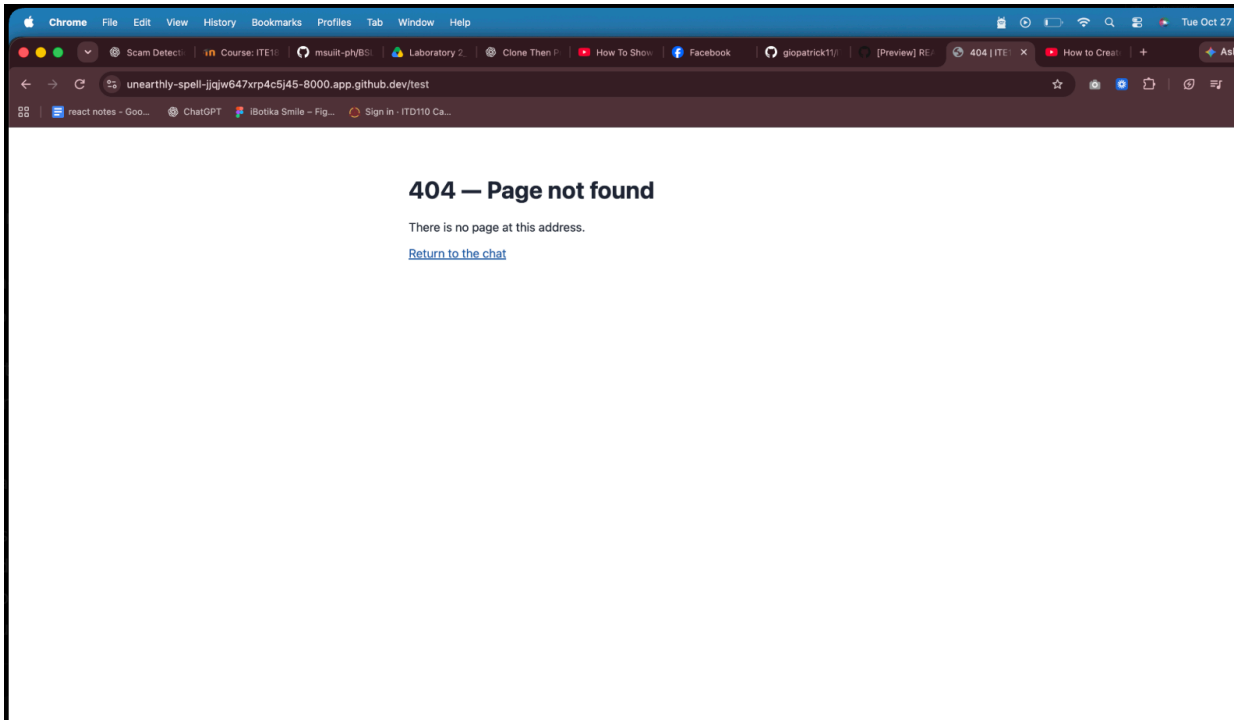


Figure 4. An undefined /test path renders the custom 404 page.

Evidence 5. WebSocket handshake

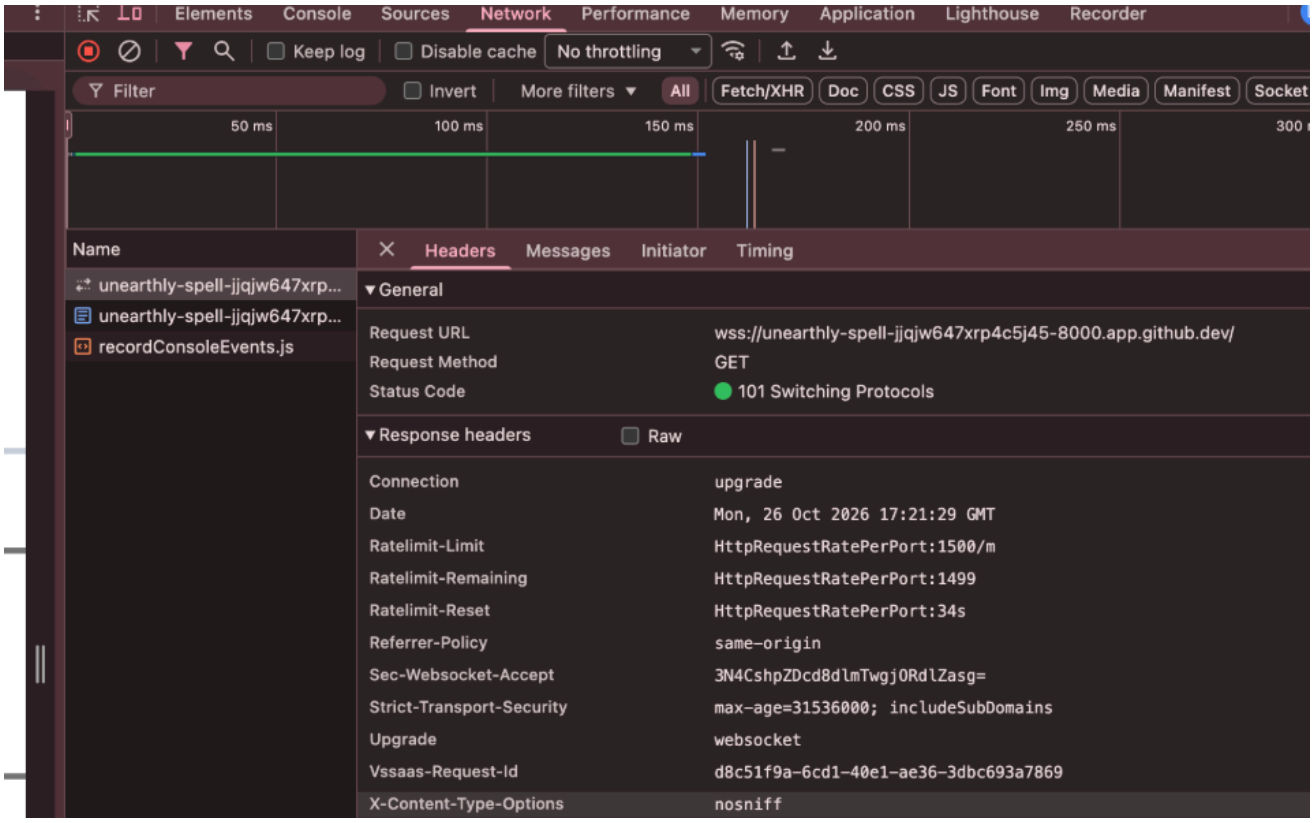


Figure 5. DevTools shows the wss:// request and HTTP 101 Switching Protocols response.